

Now, `Id` is just a simple wrapper. It doesn't really add anything. Applying `Id` to `A` is an identity since the wrapped type and the unwrapped type are totally isomorphic (we can go from one to the other and back again without any loss of information). But what is the meaning of the identity *monad*? Let's try using it in the REPL:

```
scala> Id("Hello, ") flatMap (a =>
  |   Id("monad!") flatMap (b =>
  |     Id(a + b)))
res0: Id[java.lang.String] = Id(Hello, monad!)
```

When we write the exact same thing with a for-comprehension, it might be clearer:

```
scala> for {
  |   a <- Id("Hello, ")
  |   b <- Id("monad!")
  | } yield a + b
res1: Id[java.lang.String] = Id(Hello, monad!)
```

So what is the *action* of `flatMap` for the identity monad? It's simply variable substitution. The variables `a` and `b` get bound to `"Hello, "` and `"monad!"`, respectively, and then substituted into the expression `a + b`. We could have written the same thing without the `Id` wrapper, using just Scala's own variables:

```
scala> val a = "Hello, "
a: java.lang.String = "Hello, "

scala> val b = "monad!"
b: java.lang.String = monad!

scala> a + b
res2: java.lang.String = Hello, monad!
```

Besides the `Id` wrapper, there's no difference. So now we have at least a partial answer to the question of what monads mean. We could say that monads provide a context for introducing and binding variables, and performing variable substitution.

Let's see if we can get the rest of the answer.

11.5.2 *The State monad and partial type application*

Look back at the discussion of the `State` data type in chapter 6. Recall that we implemented some combinators for `State`, including `map` and `flatMap`.

Listing 11.7 Revisiting our `State` data type

```
case class State[S, A](run: S => (A, S)) {
  def map[B](f: A => B): State[S, B] =
    State(s => {
      val (a, s1) = run(s)
      (f(a), s1)
    })
  def flatMap[B](f: A => State[S, B]): State[S, B] =
    State(s => {
```

```

    val (a, s1) = run(s)
    f(a).run(s1)
  })
}

```

It looks like `State` definitely fits the profile for being a monad. But its type constructor takes two type arguments, and `Monad` requires a type constructor of one argument, so we can't just say `Monad[State]`. But if we choose some particular `S`, then we have something like `State[S, _]`, which is the kind of thing expected by `Monad`. So `State` doesn't just have one monad instance but a whole family of them, one for each choice of `S`. We'd like to be able to partially apply `State` to where the `S` type argument is fixed to be some concrete type.

This is much like how we might partially apply a function, except at the type level. For example, we can create an `IntState` type constructor, which is an alias for `State` with its first type argument fixed to be `Int`:

```
type IntState[A] = State[Int, A]
```

And `IntState` is exactly the kind of thing that we can build a `Monad` for:

```

object IntStateMonad extends Monad[IntState] {
  def unit[A](a: => A): IntState[A] = State(s => (a, s))
  def flatMap[A,B](st: IntState[A])(f: A => IntState[B]): IntState[B] =
    st flatMap f
}

```

Of course, it would be really repetitive if we had to manually write a separate `Monad` instance for each specific state type. Unfortunately, Scala doesn't allow us to use underscore syntax to simply say `State[Int, _]` to create an anonymous type constructor like we create anonymous functions. But instead we can use something similar to lambda syntax at the type level. For example, we could have declared `IntState` directly inline like this:

```

object IntStateMonad extends
  Monad[({type IntState[A] = State[Int, A]})#IntState] {
  ...
}

```

This syntax can be jarring when you first see it. But all we're doing is declaring an anonymous type within parentheses. This anonymous type has, as one of its members, the type alias `IntState`, which looks just like before. Outside the parentheses we're then accessing its `IntState` member with the `#` syntax. Just like we can use a dot (`.`) to access a member of an object at the value level, we can use the `#` symbol to access a type member (see the "Type Projection" section of the Scala Language Specification: <http://mng.bz/u70U>).

A type constructor declared inline like this is often called a *type lambda* in Scala. We can use this trick to partially apply the `State` type constructor and declare a `StateMonad` trait. An instance of `StateMonad[S]` is then a monad instance for the given state type `S`:

```
def stateMonad[S] = new Monad[({type f[x] = State[S,x]})#f] {
  def unit[A](a: => A): State[S,A] = State(s => (a, s))
  def flatMap[A,B](st: State[S,A])(f: A => State[S,B]): State[S,B] =
    st flatMap f
}
```

← The choice of the name **f** here is arbitrary.

Again, just by giving implementations of `unit` and `flatMap`, we get implementations of all the other monadic combinators for free.



EXERCISE 11.18

Now that we have a `State` monad, you should try it out to see how it behaves. What is the meaning of `replicateM` in the `State` monad? How does `map2` behave? What about `sequence`?

Let's now look at the difference between the `Id` monad and the `State` monad. Remember that the primitive operations on `State` (besides the monadic operations `unit` and `flatMap`) are that we can read the current state with `getState` and we can set a new state with `setState`:

```
def getState[S]: State[S, S]
def setState[S](s: => S): State[S, Unit]
```

Remember that we also discovered that these combinators constitute a minimal set of primitive operations for `State`. So together with the monadic primitives (`unit` and `flatMap`) they *completely specify* everything that we can do with the `State` data type. This is true in general for monads—they all have `unit` and `flatMap`, and each monad brings its own set of additional primitive operations that are specific to it.



EXERCISE 11.19

What laws do you expect to mutually hold for `getState`, `setState`, `unit`, and `flatMap`?

What does this tell us about the meaning of the *State monad*? Let's study a simple example. The details of this code aren't too important, but notice the use of `getState` and `setState` in the `for` block.

Listing 11.8 Getting and setting state with a `for`-comprehension

```
val F = stateMonad[Int]

def zipWithIndex[A](as: List[A]): List[(Int,A)] =
  as.foldLeft(F.unit(List[(Int, A)]()))((acc,a) => for {
```

```

xs <- acc
n  <- getState
_  <- setState(n + 1)
} yield (n, a) :: xs.run(0)._1.reverse

```

This function numbers all the elements in a list using a `State` action. It keeps a state that's an `Int`, which is incremented at each step. We run the whole composite state action starting from 0. We then reverse the result since we constructed it in reverse order.¹⁰

Note what's going on with `getState` and `setState` in the for-comprehension. We're obviously getting variable binding just like in the `Id` monad—we're binding the value of each successive state action (`getState`, `acc`, and then `setState`) to variables. But there's more going on, literally *between the lines*. At each line in the for-comprehension, the implementation of `flatMap` is making sure that the current state is available to `getState`, and that the new state gets propagated to all actions that follow a `setState`.

What does the difference between the action of `Id` and the action of `State` tell us about monads in general? We can see that a chain of `flatMap` calls (or an equivalent for-comprehension) is like an imperative program with statements that assign to variables, and *the monad specifies what occurs at statement boundaries*. For example, with `Id`, nothing at all occurs except unwrapping and rewrapping in the `Id` constructor. With `State`, the most current state gets passed from one statement to the next. With the `Option` monad, a statement may return `None` and terminate the program. With the `List` monad, a statement may return many results, which causes statements that follow it to potentially run multiple times, once for each result.

The `Monad` contract doesn't specify *what* is happening between the lines, only that whatever *is* happening satisfies the laws of associativity and identity.



EXERCISE 11.20

Hard: To cement your understanding of monads, give a monad instance for the following type, and explain what it means. What are its primitive operations? What is the action of `flatMap`? What meaning does it give to monadic functions like `sequence`, `join`, and `replicateM`? What meaning does it give to the monad laws?¹¹

```

case class Reader[R, A](run: R => A)

object Reader {
  def readerMonad[R] = new Monad[({type f[x] = Reader[R,x]})#f] {
    def unit[A](a: => A): Reader[R,A]
    def flatMap[A,B](st: Reader[R,A])(f: A => Reader[R,B]): Reader[R,B]
  }
}

```

¹⁰ This is asymptotically faster than appending to the list in the loop.

¹¹ See the chapter notes for further discussion of this data type.